

GH-600 HANDS-ON WORKSHOP

SpaceRescue Mission Control

Build an agentic AI system from an empty folder to a GitHub-controlled SDLC

PowerShell | Python | OpenAI SDK | GitHub | MCP

ROVER-07 has a dangerous thermal event on Mars. Learners progressively build the mission-control agent that plans, uses tools, remembers, evaluates, coordinates, applies guardrails, and participates in a controlled GitHub workflow.

1. Workshop Map

The current GH-600 study guide has six domains: architecture and SDLC; tool use and environment interaction; memory/state/execution; evaluation/error analysis/tuning; multi-agent coordination; and guardrails/accountability. This workshop uses one small application to cover all six.

- Start from an empty folder.
- Use PowerShell throughout.
- Use the real OpenAI Python SDK in Stage 2.
- Every changed file is provided as a complete file.
- Keep the model separate from validation, authorization and execution.

Prerequisites

- Windows + PowerShell
- Python 3.11+
- Git and VS Code
- GitHub repository access
- OpenAI Developer Platform API key and applicable project credits

API key

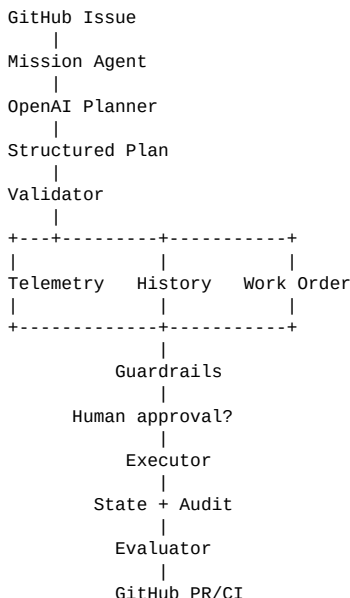
PowerShell

```
$env:OPENAI_API_KEY = "YOUR_API_KEY"
if ($env:OPENAI_API_KEY) { "API key configured" } else { "API key missing" }
```

Teach point: Never commit an API key. Use environment variables or a secret manager. Rotate a key immediately if it is exposed.

2. Final Architecture

Architecture



Teach point: The model proposes. Code validates. Policy authorizes. Tools execute. State records. Evaluation measures. GitHub reviews.

3. Stage 1 — Deterministic Baseline

GH-600: define goal, input, output, success criteria, boundaries and tests before adding autonomy.

PowerShell setup

```

mkdir space-rescue-agent
cd space-rescue-agent
git init
python -m venv .venv
.\.venv\Scripts\Activate.ps1
New-Item -ItemType Directory -Force app,tests,tools,memory,evaluation,guardrails,.github\workflows
New-Item -ItemType File -Force app\__init__.py,app\main.py,app\mission.py,tests\test_mission.py

```

app/mission.py — complete file

```

from dataclasses import dataclass

@dataclass
class MissionRequest:
    request_id: str
    rover_id: str
    message: str

def classify_incident(message: str) -> str:
    text = message.lower()
    if "overheat" in text or "temperature" in text:
        return "thermal"
    if "wheel" in text or "motor" in text:
        return "mobility"
    if "battery" in text or "power" in text:
        return "power"
    return "unknown"

def create_plan(request: MissionRequest) -> dict:
    category = classify_incident(request.message)
    actions = {
        "thermal": ["check_temperature", "reduce_rover_load", "request_engineer_review"],
        "mobility": ["check_wheel_telemetry", "slow_rover", "request_engineer_review"],
        "power": ["check_battery", "enter_power_save", "request_engineer_review"],
        "unknown": ["collect_more_information", "request_engineer_review"],
    }
    return {"request_id": request.request_id, "rover_id": request.rover_id,
            "category": category, "actions": actions[category]}

```

app/main.py — complete file

```

from app.mission import MissionRequest, create_plan

def main():
    request=MissionRequest(
        "INC-001", "ROVER-07",
        "ROVER-07 reports wheel motor overheating near sector A3."
    )
    print(create_plan(request))

if __name__=="__main__":
    main()

```

tests/test_mission.py — complete file

```

import unittest
from app.mission import MissionRequest, classify_incident, create_plan

class MissionTests(unittest.TestCase):
    def test_thermal(self):
        self.assertEqual(classify_incident("motor overheating"), "thermal")
    def test_power(self):
        self.assertEqual(classify_incident("battery power low"), "power")
    def test_unknown(self):
        self.assertEqual(classify_incident("strange noise"), "unknown")
    def test_plan(self):
        p=create_plan(MissionRequest("1", "ROVER-07", "motor overheating"))
        self.assertEqual(p["category"], "thermal")
        self.assertIn("check_temperature", p["actions"])

if __name__=="__main__":
    unittest.main()

```

Run + commit

```

python -m app.main
python -m unittest discover -v
git add .
git commit -m "Create SpaceRescue baseline"

```

Teach point: This is the control case. It is deliberately deterministic so learners can compare it with later model-driven behavior.

4. Stage 2 — Real OpenAI Planner

GH-600: separate planning from execution, use structured plans, validate them, and block unsafe actions.

The official OpenAI quickstart uses the Python SDK with the Responses API. Structured Outputs can enforce a JSON Schema.

Install SDK

```
pip install --upgrade openai
pip freeze > requirements.txt
```

app/planner.py — complete file

```
import json
from openai import OpenAI

ALLOWED_ACTIONS={
    "check_temperature", "check_wheel_telemetry", "check_battery",
    "collect_more_information", "request_engineer_review",
    "reduce_rover_load", "slow_rover", "enter_power_save"
}

PLAN_SCHEMA={
    "type": "object",
    "properties": {
        "category": {"type": "string", "enum": ["thermal", "mobility", "power", "unknown"]},
        "priority": {"type": "string", "enum": ["low", "medium", "high"]},
        "actions": {"type": "array", "items": {"type": "string", "enum": sorted(ALLOWED_ACTIONS)}},
        "rationale": {"type": "string"}
    },
    "required": ["category", "priority", "actions", "rationale"],
    "additionalProperties": False
}

def validate_plan(plan, request_id, rover_id):
    errors=[]
    if plan.get("priority") not in {"low", "medium", "high"}:
        errors.append("invalid priority")
    if not isinstance(plan.get("actions"), list):
        errors.append("actions must be a list")
    else:
        for a in plan["actions"]:
            if a not in ALLOWED_ACTIONS:
                errors.append(f"unsupported action: {a}")
    if not plan.get("rationale"):
        errors.append("rationale is required")
    if errors:
        raise ValueError(json.dumps(errors, indent=2))
    return {"request_id": request_id, "rover_id": rover_id, **plan}

def create_plan(request_id, rover_id, message):
    client=OpenAI()
    instructions="""
You are the SpaceRescue planning agent.
Analyze a rover incident and propose a safe plan.
Use only schema actions. Use high priority for credible thermal danger.
Prefer information gathering when evidence is weak.
Never invent actions. The plan is only a proposal.
"""
    response=client.responses.create(
        model="gpt-5",
        instructions=instructions,
        input=f"Request ID: {request_id}\nRover ID: {rover_id}\nIncident: {message}",
        text={"format": {
            "type": "json_schema", "name": "mission_plan", "strict": True,
            "schema": PLAN_SCHEMA
        }},
    )
    return validate_plan(json.loads(response.output_text), request_id, rover_id)
```

app/main.py — complete file

```
from app.planner import create_plan

def main():
    plan=create_plan(
        "INC-001", "ROVER-07",
        "ROVER-07 reports a sudden wheel-motor temperature spike near sector A3."
    )
    print("\nVALIDATED AI PLAN")
    for k,v in plan.items():
        print(f"{k}: {v}")

if __name__=="__main__":
    main()
```

Run + negative test

```
python -m app.main
python -c "from app.planner import validate_plan; print(validate_plan({'category': 'thermal', 'priority': 'high', 'actions': ['delete_mission']}, 'INC-001', 'ROVER-07'))"
git add .
git commit -m "Add real OpenAI planning agent"
```

Teach point: Structured output controls shape, not authority. Treat model output as untrusted input; application validation owns business rules and capability boundaries.

5. Stage 3 — Tools and MCP

GH-600: tools, permissions, MCP, safe execution, errors, retries/rollback/escalation concepts and traceability.

tools/telemetry.py — complete file

```
TELEMETRY={"ROVER-07":{"temperature_c":92,"battery_pct":61,"wheel_motor_rpm":740,"sector":"A3"}}

def get_telemetry(rover_id):
    if rover_id not in TELEMETRY:
        raise ValueError(f"Unknown rover: {rover_id}")
    return TELEMETRY[rover_id].copy()
```

tools/history.py — complete file

```
HISTORY={"ROVER-07":[
    {"incident":"wheel vibration","sector":"A3","status":"resolved"},
    {"incident":"temperature warning","sector":"A2","status":"closed"},
]}

def get_history(rover_id):
    return list(HISTORY.get(rover_id, []))
```

tools/work_order.py — complete file

```
def create_work_order(rover_id, reason, approved):
    if not approved:
        raise PermissionError("Creating a work order requires explicit human approval.")
    return {"status":"created","rover_id":rover_id,"reason":reason}
```

app/executor.py — complete file

```
from tools.telemetry import get_telemetry
from tools.work_order import create_work_order

def execute_action(action, rover_id, approved=False):
    if action=="check_temperature":
        return {"temperature_c":get_telemetry(rover_id)["temperature_c"]}
    if action=="check_wheel_telemetry":
        x=get_telemetry(rover_id)
        return {"wheel_motor_rpm":x["wheel_motor_rpm"],"sector":x["sector"]}
    if action=="check_battery":
        return {"battery_pct":get_telemetry(rover_id)["battery_pct"]}
    if action=="request_engineer_review":
        return create_work_order(rover_id,"Engineer review requested.",approved)
    raise ValueError(f"Unsupported action: {action}")

def execute_plan(plan, approved=False):
    return [{"action":a,"result":execute_action(a,plan["rover_id"],approved)}
            for a in plan["actions"]]
```

Test

```
python -c "from tools.telemetry import get_telemetry; print(get_telemetry('ROVER-07'))"
python -c "from tools.work_order import create_work_order; print(create_work_order('ROVER-07','review',False))"
```

Teach point: MCP standardizes tool interaction; it does not replace authorization. Discuss MCP servers, GitHub remote MCP, registries, allow lists and environment scope.

Commit

```
git add .
git commit -m "Add controlled mission tools"
```

6. Stage 4 — Memory, State and Resume

GH-600: memory strategies, durable progress, resume, stale context and conflict handling.

memory/state.py — complete file

```
import json
from pathlib import Path

STATE_FILE=Path("memory/mission_state.json")

def load_state():
    if not STATE_FILE.exists(): return {}
    return json.loads(STATE_FILE.read_text())

def save_state(state):
    STATE_FILE.write_text(json.dumps(state,indent=2))

def reset_state():
    if STATE_FILE.exists(): STATE_FILE.unlink()

def mark_completed(request_id,action,result):
```

```

state=load_state()
mission=state.setdefault(request_id,{"completed_actions":[],"decisions":{}})
if action not in mission["completed_actions"]:
    mission["completed_actions"].append(action)
mission["last_result"]=result
save_state(state)
return mission

```

app/resume.py — complete file

```

from memory.state import load_state

def remaining_actions(request_id,actions):
    state=load_state()
    completed=set(state.get(request_id,{}).get("completed_actions",[]))
    return [a for a in actions if a not in completed]

```

Test

```

python -c "from memory.state import mark_completed; print(mark_completed('INC-001','check_temperature',{ 'temperature_c':92}))"
python -c "from app.resume import remaining_actions; print(remaining_actions('INC-001',['check_temperature','reduce_rover_load','request_engineer_review']))"
git add .
git commit -m "Add durable mission state"

```

Teach point: Context is what is available now; state records progress; memory is intentionally retained information. Production systems need scope, pruning, expiration and reset rules.

7. Stage 5 — Evaluation, Error Analysis and Tuning

GH-600: success criteria, evaluation signals, root-cause analysis and tuning.

evaluation/evaluator.py — complete file

```

ALLOWED={
    "check_temperature","check_wheel_telemetry","check_battery",
    "collect_more_information","request_engineer_review",
    "reduce_rover_load","slow_rover","enter_power_save"
}

def evaluate_plan(plan):
    checks={
        "has_request_id":bool(plan.get("request_id")),
        "has_rover_id":bool(plan.get("rover_id")),
        "valid_priority":plan.get("priority") in {"low","medium","high"},
        "all_actions_allowed":all(a in ALLOWED for a in plan.get("actions",[])),
        "has_rationale":bool(plan.get("rationale"))
    }
    passed=sum(checks.values())
    return {"passed":passed==len(checks),"score":f"{passed}/{len(checks)}","checks":checks}

```

evaluation/evaluation_cases.json — complete file

```

[
    {"name":"thermal incident","message":"wheel motor overheating","expected_category":"thermal","expected_priority":"high"},
    {"name":"unknown incident","message":"strange noise","expected_category":"unknown","expected_priority":"medium"}
]

```

Run

```

python -c "from app.planner import create_plan; from evaluation.evaluator import evaluate_plan; p=create_plan('INC-001','ROVER-07','wheel_motor_overheating')"
git add .
git commit -m "Add agent evaluation"

```

Teach point: Failure classes: reasoning error, tool misuse, context issue, environment failure, policy failure. Tune instructions, tools, memory or workflow based on evidence.

8. Stage 6 — Multi-Agent Coordination

GH-600: orchestration, isolation, conflict detection, observability, recovery and lifecycle.

app/agents.py — complete file

```

from tools.telemetry import get_telemetry
from evaluation.evaluator import evaluate_plan

def diagnoser(rover_id):
    t=get_telemetry(rover_id)
    return {"agent":"diagnoser","finding":"thermal-risk" if t["temperature_c"]>=90 else "normal","evidence":t}

def safety_agent(plan):
    risky={"reduce_rover_load","slow_rover","enter_power_save"}
    approval=any(a in risky for a in plan["actions"])
    return {"agent":"safety","requires_approval":approval,
            "reason":"Operational state change detected." if approval else "Read-only or review actions only."}

def coordinator(plan):

```

```
return {"plan":plan,"diagnosis":diagnoser(plan["rover_id"]),
        "safety":safety_agent(plan),"evaluation":evaluate_plan(plan)}
```

Run

```
python -c "from app.planner import create_plan; from app.agents import coordinator; p=create_plan('INC-001','ROVER-07','wheel motor overh
git add .
git commit -m "Add specialist agent coordination"
```

Teach point: Ask what happens when agents disagree. Require an explicit conflict-resolution policy; do not leave it to an implicit model decision.

9. Stage 7 — Guardrails, Approval and Audit

GH-600: autonomy levels, least privilege, human-in-the-loop, blocking rules and accountability.

guardrails/policy.py — complete file

```
READ_ONLY={"check_temperature","check_wheel_telemetry","check_battery"}
MEDIUM={"collect_more_information","request_engineer_review"}
HIGH={"reduce_rover_load","slow_rover","enter_power_save"}

def classify(action):
    if action in READ_ONLY: return "low"
    if action in MEDIUM: return "medium"
    if action in HIGH: return "high"
    return "blocked"

def authorize(action,human_approved=False):
    risk=classify(action)
    if risk=="blocked":
        return {"allowed":False,"risk":risk,"reason":"Unknown action."}
    if risk=="high" and not human_approved:
        return {"allowed":False,"risk":risk,"reason":"Human approval required."}
    return {"allowed":True,"risk":risk,"reason":"Policy passed."}
```

app/audit.py — complete file

```
from datetime import datetime

def audit(event,details):
    print("AUDIT",{
        "timestamp":datetime.utcnow().isoformat()+"Z",
        "event":event,
        "details":details,
    })
```

Test

```
python -c "from guardrails.policy import authorize; print(authorize('check_temperature')); print(authorize('slow_rover')); print(authoriz
git add .
git commit -m "Add risk based guardrails and audit"
```

Teach point: Prompts are not security boundaries. Enforce authorization in code. Use risk-based autonomy and audit consequential actions.

10. Stage 8 — GitHub Agentic SDLC

GH-600: repository/branch scope, CI, custom instructions, inspectable artifacts, PRs and human review.

.github/copilot-instructions.md — complete file

```
# SpaceRescue Agent Instructions

## Mission
Keep the SpaceRescue application small, testable, understandable and safe.

## Rules
- Do not modify files outside task scope.
- Do not invent tools.
- Never remove safety checks or audit logging.
- Add/update tests for behavior changes.
- Prefer small changes.
- Explain assumptions in the pull request.
- Never merge a pull request.
- Never expose secrets.
- High-risk operational actions require human approval.

## Definition of Done
- Tests pass.
- Evaluation cases pass.
- Guardrails remain enabled.
- Audit behavior remains intact.
- Change is reviewable through a pull request.
```

.github/workflows/test.yml — complete file

```
name: SpaceRescue Tests

on:
  pull_request:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
      - run: python -m unittest discover -v
```

Push

```
git add .
git commit -m "Add GitHub agentic SDLC controls"
git branch -M main
git remote add origin YOUR_REPOSITORY_URL
git push -u origin main
```

Capstone issue

GitHub issue

Add support for radiation spike incidents.

```
Requirements:
- classify radiation incidents
- assign high priority
- add radiation telemetry
- add tests
- do not weaken existing safety rules
- explain the change in the pull request
```

Target lifecycle: issue -> understand -> plan -> validate -> branch -> implement -> test -> evaluate -> CI -> PR -> human review.

Teach point: Agentic SDLC is controlled autonomy, not unrestricted automation.

11. Instructor Final Demo

- Show deterministic baseline.
- Run real OpenAI planning.
- Show structured output.
- Attempt unsupported action; show rejection.
- Read telemetry through a tool.
- Attempt high-risk action without approval; show block.
- Approve and show audit.
- Restart and show state resume.
- Create an evaluation failure and classify it.
- Finish with GitHub issue -> PR -> CI -> human review.

12. GH-600 Coverage Map

Domain	Hands-on
Architecture & SDLC	Stages 1, 2, 8
Tool use & environment	Stage 3
Memory/state/execution	Stage 4
Evaluation/error/tuning	Stage 5

Multi-agent coordination	Stage 6
Guardrails/accountability	Stage 7

13. Production Pitfalls

- LLM treated as authority.
- Arbitrary model-generated code execution.
- Authorization only in prompts.
- Every agent gets every tool.
- Unscoped memory.
- Unsafe retries of non-idempotent actions.
- No audit trail.
- No conflict policy.
- No human control for consequential changes.
- Secrets committed to Git.

14. Learner Checklist

- Explain LLM vs agent vs tool vs MCP.
- Separate planning from execution.
- Validate structured plans.
- Use least privilege.
- Distinguish context, memory and state.
- Define evaluation signals and failure classes.
- Explain multi-agent orchestration.
- Define autonomy and approval levels.
- Explain GitHub branch/PR/CI controls.
- Identify observability and audit artifacts.

15. Official References

OpenAI Quickstart: <https://platform.openai.com/docs/quickstart/make-your-first-api-request>

OpenAI Responses/Structured Output reference:
<https://platform.openai.com/docs/api-reference/responses-streaming/response/refusal>

Microsoft GH-600 Study Guide: <https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/gh-600>

Microsoft GH-600 Course: <https://learn.microsoft.com/en-us/training/courses/gh-600t00>

16. Optional Extensions

- Replace local tools with MCP.
- Demonstrate GitHub remote MCP and allow lists.
- Add retry/rollback/escalation.
- Emit plan/tool/result/policy/approval traces.
- Add CodeQL, Dependabot and secret scanning.
- Create custom Copilot agent instructions.
- Add model-based evaluation.

- Build controlled issue-to-PR automation.

Microsoft describes GH-600 as an intermediate one-day course focused on developing, deploying and managing agentic AI systems within GitHub-based SDLC workflows, including GitHub Copilot, MCP and agent customization.